

SOCKET PROGRAMMING PRACTICE ASSIGNMENTS

Question 1 – Hello Client-Server (Beginner)

Problem Statement

Develop a TCP client-server application where:

- The client sends the message “Hello Server”.
- The server receives the message and displays it.
- The server replies with “Hello Client”.
- The client displays the received response.

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
{
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;
    char buffer[100];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0)
    {
        perror("Socket creation failed");
        return 1;
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&server_addr,
             sizeof(server_addr)) < 0)
    {
        perror("Bind failed");
        return 1;
    }

    listen(server_fd, 5);

    printf("Server started...\n");
```

```

printf("Waiting for client...\n");

addr_size = sizeof(client_addr);
client_fd = accept(server_fd,
                  (struct sockaddr*)&client_addr,
                  &addr_size);

if (client_fd < 0)
{
    perror("Accept failed");
    return 1;
}

recv(client_fd, buffer, sizeof(buffer), 0);

printf("Message from client: %s\n", buffer);

strcpy(buffer, "Hello Client");
send(client_fd, buffer, strlen(buffer) + 1, 0);

printf("Reply sent to client.\n");

close(client_fd);
close(server_fd);

return 0;
}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
{
    int sock;
    struct sockaddr_in server_addr;
    char buffer[100];

    sock = socket(AF_INET, SOCK_STREAM, 0);

    if (sock < 0)
    {
        perror("Socket creation failed");
        return 1;
    }

```

```

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

if (connect(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0)
{
    perror("Connection failed");
    return 1;
}

printf("Connected to server.\n");

strcpy(buffer, "Hello Server");

send(sock, buffer, strlen(buffer) + 1, 0);

printf("Message sent: %s\n", buffer);

recv(sock, buffer, sizeof(buffer), 0);

printf("Server response: %s\n", buffer);

close(sock);

return 0;
}

```

SCREENSHOTS

```

crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc server.c -o server
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./server
Server started...
Waiting for client...
Message from client: Hello Server
Reply sent to client.

```

Fig 1.1: Server Terminal

```

crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./client
Connected to server.
Message sent: Hello Server
Server response: Hello Client

```

Fig 1.2: Client Terminal

Question 2 – Arithmetic Server

Problem Statement

Develop a TCP client-server application where:

- The client inputs two integers.
- The client sends both numbers to the server.

- The server calculates:
 - Addition
 - Subtraction
 - Multiplication
 - Division
- The server sends all results back.
- The client displays the received results.

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
{
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;

    int a, b;
    int addition, subtraction, multiplication;
    float division;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0)
    {
        perror("Socket creation failed");
        return 1;
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&server_addr,
        sizeof(server_addr));

    listen(server_fd, 5);

    printf("Server started...\n");
    printf("Waiting for client...\n");

    addr_size = sizeof(client_addr);
```

```

client_fd = accept(server_fd,
                   (struct sockaddr *)&client_addr,
                   &addr_size);

recv(client_fd, &a, sizeof(a), 0);
recv(client_fd, &b, sizeof(b), 0);

addition = a + b;
subtraction = a - b;
multiplication = a * b;

if (b != 0)
    division = (float)a / b;
else
    division = 0;

send(client_fd, &addition, sizeof(addition), 0);
send(client_fd, &subtraction, sizeof(subtraction), 0);
send(client_fd, &multiplication, sizeof(multiplication), 0);
send(client_fd, &division, sizeof(division), 0);

printf("\nResults:\n");
printf("Addition      : %d\n", addition);
printf("Subtraction   : %d\n", subtraction);
printf("Multiplication : %d\n", multiplication);
printf("Division      : %.2f\n", division);

close(client_fd);
close(server_fd);

return 0;
}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
{
    int sock;
    struct sockaddr_in server_addr;

    int a, b;
    int addition, subtraction, multiplication;
    float division;

```

```

sock = socket(AF_INET, SOCK_STREAM, 0);

if (sock < 0)
{
    perror("Socket creation failed");
    return 1;
}

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

if (connect(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0)
{
    perror("Connection failed");
    return 1;
}

printf("Connected to server.\n");

printf("Enter first integer: ");
scanf("%d", &a);

printf("Enter second integer: ");
scanf("%d", &b);

send(sock, &a, sizeof(a), 0);
send(sock, &b, sizeof(b), 0);

recv(sock, &addition, sizeof(addition), 0);
recv(sock, &subtraction, sizeof(subtraction), 0);
recv(sock, &multiplication, sizeof(multiplication), 0);
recv(sock, &division, sizeof(division), 0);

printf("\n----- Arithmetic Results ----- \n");
printf("Addition      : %d\n", addition);
printf("Subtraction   : %d\n", subtraction);
printf("Multiplication : %d\n", multiplication);
printf("Division      : %.2f\n", division);

close(sock);

return 0;
}

```

SCREENSHOTS

```
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc server.c -o server
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./server
Server started...
Waiting for client...

Results:
Addition      : 30
Subtraction   : -10
Multiplication: 200
Division      : 0.50
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ^C
```

Fig 2.1: Server Terminal

```
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc client.c -o client
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./client
Connected to server.
Enter first integer: 10
Enter second integer: 20

---- Arithmetic Results ----
Addition      : 30
Subtraction   : -10
Multiplication: 200
Division      : 0.50
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc client.c -o client
```

Fig 2.2: Client Terminal

Question 3 – String Processing Server

Problem Statement

Develop a TCP client-server application where:

- The client inputs a string.
- The server performs the following operations:
 - Find the string length
 - Convert the string to uppercase
 - Reverse the string
- The server sends the processed results.
- The client displays all outputs.

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
```

```
#define PORT 8080
```

```
int main()
{
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;
```

```

char str[200];
char uppercase[200];
char reverse[200];

int length;
int i;

server_fd = socket(AF_INET, SOCK_STREAM, 0);

if (server_fd < 0)
{
    perror("Socket creation failed");
    return 1;
}

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

bind(server_fd, (struct sockaddr *)&server_addr,
      sizeof(server_addr));

listen(server_fd, 5);

printf("Server started...\n");
printf("Waiting for client...\n");

addr_size = sizeof(client_addr);

client_fd = accept(server_fd,
                  (struct sockaddr *)&client_addr,
                  &addr_size);

recv(client_fd, str, sizeof(str), 0);

length = strlen(str);

/* Convert to uppercase */
for (i = 0; i < length; i++)
    uppercase[i] = toupper(str[i]);

uppercase[length] = '\0';

/* Reverse string */
for (i = 0; i < length; i++)
    reverse[i] = str[length - i - 1];

reverse[length] = '\0';

```



```

send(client_fd, &length, sizeof(length), 0);
send(client_fd, uppercase, sizeof(uppercase), 0);
send(client_fd, reverse, sizeof(reverse), 0);

printf("\nResults:\n");
printf("Length   : %d\n", length);
printf("Uppercase : %s\n", uppercase);
printf("Reverse  : %s\n", reverse);

close(client_fd);
close(server_fd);

return 0;
}
client.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
{
    int sock;
    struct sockaddr_in server_addr;

    char str[200];
    char uppercase[200];
    char reverse[200];
    int length;

    sock = socket(AF_INET, SOCK_STREAM, 0);

    if (sock < 0)
    {
        perror("Socket creation failed");
        return 1;
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    if (connect(sock, (struct sockaddr *)&server_addr,
        sizeof(server_addr)) < 0)
    {
        perror("Connection failed");
        return 1;
    }

```

```

    }

    printf("Connected to server.\n");

    printf("Enter a string: ");
    scanf(" %[^\\n]", str);

    send(sock, str, sizeof(str), 0);

    recv(sock, &length, sizeof(length), 0);
    recv(sock, uppercase, sizeof(uppercase), 0);
    recv(sock, reverse, sizeof(reverse), 0);

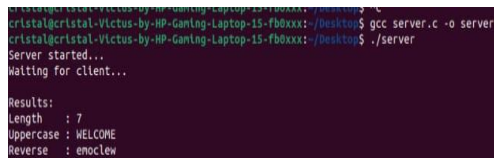
    printf("\\n----- String Processing -----\\n");
    printf("Length   : %d\\n", length);
    printf("Uppercase : %s\\n", uppercase);
    printf("Reverse  : %s\\n", reverse);

    close(sock);

    return 0;
}

```

SCREENSHOTS



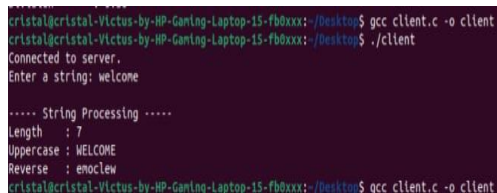
```

cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc server.c -o server
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./server
Server started...
Waiting for client...

Results:
Length   : 7
Uppercase : WELCOME
Reverse  : enoclew

```

Fig 3.1: Server Terminal



```

cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc client.c -o client
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./client
Connected to server.
Enter a string: welcome

----- String Processing -----
Length   : 7
Uppercase : WELCOME
Reverse  : enoclew
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc client.c -o client

```

Fig 3.2: Client Terminal

Question 4 – Array Processing Server

Problem Statement

Develop a TCP client-server application where:

- The client inputs N integers.

- The array is sent to the server.
- The server computes:
 - Sum
 - Average
 - Maximum Element
 - Minimum Element
- The server sends the results.
- The client displays them.

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX 100

int main()
{
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;

    int n;
    int arr[MAX];
    int sum = 0;
    int maximum, minimum;
    float average;
    int i;

    /* Create socket */
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0)
    {
        perror("Socket creation failed");
        exit(1);
    }

    /* Configure server */
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    /* Bind */
```

```

if (bind(server_fd, (struct sockaddr *)&server_addr,
        sizeof(server_addr)) < 0)
{
    perror("Bind failed");
    close(server_fd);
    exit(1);
}

/* Listen */
listen(server_fd, 5);

printf("Server started...\n");
printf("Waiting for client connection...\n");

/* Accept client */
addr_size = sizeof(client_addr);

client_fd = accept(server_fd,
                  (struct sockaddr *)&client_addr,
                  &addr_size);

if (client_fd < 0)
{
    perror("Accept failed");
    close(server_fd);
    exit(1);
}

printf("Client connected.\n");

/* Receive number of elements */
recv(client_fd, &n, sizeof(n), 0);

/* Receive array */
recv(client_fd, arr, sizeof(int) * n, 0);

/* Initialize maximum and minimum */
maximum = arr[0];
minimum = arr[0];

/* Calculate results */
for (i = 0; i < n; i++)
{
    sum += arr[i];

    if (arr[i] > maximum)
        maximum = arr[i];

    if (arr[i] < minimum)
        minimum = arr[i];
}

```

```

}

average = (float)sum / n;

/* Send results */
send(client_fd, &sum, sizeof(sum), 0);
send(client_fd, &average, sizeof(average), 0);
send(client_fd, &maximum, sizeof(maximum), 0);
send(client_fd, &minimum, sizeof(minimum), 0);

/* Display results on server */
printf("\nResults sent to client.\n");
printf("Sum    : %d\n", sum);
printf("Average : %.2f\n", average);
printf("Maximum : %d\n", maximum);
printf("Minimum : %d\n", minimum);

close(client_fd);
close(server_fd);

return 0;
}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX 100

int main()
{
    int sock;
    struct sockaddr_in server_addr;

    int n;
    int arr[MAX];
    int sum, maximum, minimum;
    float average;
    int i;

    /* Create socket */
    sock = socket(AF_INET, SOCK_STREAM, 0);

    if (sock < 0)
    {
        perror("Socket creation failed");
        exit(1);
    }
}

```

```

/* Configure server */
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

/* Connect */
if (connect(sock,
            (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0)
{
    perror("Connection failed");
    close(sock);
    exit(1);
}

printf("Connected to server.\n");

/* Input number of elements */
printf("\nEnter number of elements: ");
scanf("%d", &n);

printf("Enter %d integers:\n", n);

for (i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
}

/* Send number of elements */
send(sock, &n, sizeof(n), 0);

/* Send array */
send(sock, arr, sizeof(int) * n, 0);

/* Receive results */
recv(sock, &sum, sizeof(sum), 0);
recv(sock, &average, sizeof(average), 0);
recv(sock, &maximum, sizeof(maximum), 0);
recv(sock, &minimum, sizeof(minimum), 0);

/* Display results */
printf("\n----- Array Analysis ----- \n");
printf("Sum    : %d\n", sum);
printf("Average : %.2f\n", average);
printf("Maximum : %d\n", maximum);
printf("Minimum : %d\n", minimum);

close(sock);

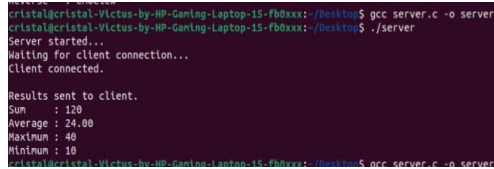
```

```

    return 0;
}

```

SCREENSHOTS



```

crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc server.c -o server
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./server
Server started...
Waiting for client connection...
Client connected.
Results sent to client.
Sum      : 120
Average  : 24.00
Maximum  : 40
Minimum  : 10
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc server.c -o server

```

Fig 4.1: Server Terminal



```

crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc client.c -o client
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./client
Connected to server.
Enter number of elements: 5
Enter 5 integers:
20
30
40
10
20

----- Array Analysis -----
Sum      : 120
Average  : 24.00
Maximum  : 40
Minimum  : 10
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc client.c -o client

```

Fig 4.2: Client Terminal

Question 5 – Matrix Analyzer

Problem Statement

Develop a TCP client-server application where:

- The client inputs a square matrix of order N.
- The matrix is sent to the server.
- The server determines:
 - Matrix Type
 - Sum of Matrix Elements
 - Trace of Matrix
- The server sends the results.
- The client displays the received information.

Matrix Types

- Diagonal Matrix
- Upper Triangular Matrix
- Lower Triangular Matrix
- General Matrix

server.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

```

```

#define PORT 8080

```

```

#define MAX 100

int main()
{
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;

    int n;
    int matrix[MAX][MAX];
    int sum = 0, trace = 0;
    int i, j;

    /* Create socket */
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0)
    {
        perror("Socket creation failed");
        exit(1);
    }

    /* Configure server address */
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    /* Bind socket */
    if (bind(server_fd, (struct sockaddr *)&server_addr,
        sizeof(server_addr)) < 0)
    {
        perror("Bind failed");
        close(server_fd);
        exit(1);
    }

    /* Listen */
    if (listen(server_fd, 5) < 0)
    {
        perror("Listen failed");
        close(server_fd);
        exit(1);
    }

    printf("Server started...\n");
    printf("Waiting for client connection...\n");

    /* Accept client */
    addr_size = sizeof(client_addr);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr,

```



```

        &addr_size);

if (client_fd < 0)
{
    perror("Accept failed");
    close(server_fd);
    exit(1);
}

printf("Client connected.\n");

/* Receive matrix order */
recv(client_fd, &n, sizeof(n), 0);

/* Receive matrix */
recv(client_fd, matrix, sizeof(int) * n * n, 0);

/* Calculate sum and trace */
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        sum += matrix[i][j];

        if (i == j)
            trace += matrix[i][j];
    }
}

/* Determine matrix type */
int is_diagonal = 1;
int is_upper = 1;
int is_lower = 1;

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        /* Diagonal matrix */
        if (i != j && matrix[i][j] != 0)
            is_diagonal = 0;

        /* Upper triangular matrix */
        if (i > j && matrix[i][j] != 0)
            is_upper = 0;

        /* Lower triangular matrix */
        if (i < j && matrix[i][j] != 0)
            is_lower = 0;
    }
}

```

```

    }

    char matrix_type[30];

    if (is_diagonal)
        strcpy(matrix_type, "Diagonal Matrix");
    else if (is_upper)
        strcpy(matrix_type, "Upper Triangular Matrix");
    else if (is_lower)
        strcpy(matrix_type, "Lower Triangular Matrix");
    else
        strcpy(matrix_type, "General Matrix");

    /* Send results to client */
    send(client_fd, matrix_type, sizeof(matrix_type), 0);
    send(client_fd, &sum, sizeof(sum), 0);
    send(client_fd, &trace, sizeof(trace), 0);

    printf("\nResults sent to client.\n");
    printf("Matrix Type : %s\n", matrix_type);
    printf("Sum      : %d\n", sum);
    printf("Trace    : %d\n", trace);

    close(client_fd);
    close(server_fd);

    return 0;
}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX 100

int main()
{
    int sock;
    struct sockaddr_in server_addr;

    int n;
    int matrix[MAX][MAX];
    int sum, trace;
    char matrix_type[30];

    int i, j;

```

```

/* Create socket */
sock = socket(AF_INET, SOCK_STREAM, 0);

if (sock < 0)
{
    perror("Socket creation failed");
    exit(1);
}

/* Configure server address */
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

/* Connect to server */
if (connect(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0)
{
    perror("Connection failed");
    close(sock);
    exit(1);
}

printf("Connected to server.\n");

/* Input matrix order */
printf("\nEnter the order of matrix: ");
scanf("%d", &n);

printf("Enter the matrix elements:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &matrix[i][j]);
    }
}

/* Send matrix order */
send(sock, &n, sizeof(n), 0);

/* Send matrix */
send(sock, matrix, sizeof(int) * n * n, 0);

/* Receive results */
recv(sock, matrix_type, sizeof(matrix_type), 0);
recv(sock, &sum, sizeof(sum), 0);
recv(sock, &trace, sizeof(trace), 0);

```

```

/* Display results */
printf("\n----- Matrix Analysis -----\\n");
printf("Matrix Type : %s\\n", matrix_type);
printf("Sum      : %d\\n", sum);
printf("Trace     : %d\\n", trace);

close(sock);

return 0;
}

```

SCREENSHOTS



```

Connection failed: Connection refused
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ gcc server.c -o server
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./server
Server started...
Waiting for client connection...
Client connected.
Results sent to client.
Matrix Type : Upper Triangular Matrix
Sum      : 0
Trace    : 1
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$

```

Fig 5.1: Server Terminal



```

crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$ ./client
Connected to server.
Enter the order of matrix: 3
Enter the matrix elements:
1 2 3
3
4 5
6 7 8
----- Matrix Analysis -----
Matrix Type : Upper Triangular Matrix
Sum      : 0
Trace    : 1
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx:~/Desktop$

```

Fig 5.2: Client Terminal